

VELORA — Module Auth

**Register · Login · Logout · Refresh · Forgot Password
· Reset Password**

Folder Structure

```
app/Modules/Auth/  
├── Controllers/  
│   └── AuthController.php  
├── DTOs/  
│   ├── RegisterDTO.php  
│   └── LoginDTO.php  
├── Requests/  
│   ├── RegisterRequest.php  
│   ├── LoginRequest.php  
│   ├── ForgotPasswordRequest.php  
│   └── ResetPasswordRequest.php  
├── Resources/  
│   └── AuthResource.php  
├── Services/  
│   └── AuthService.php  
├── Events/  
│   └── UserRegistered.php  
├── Listeners/  
│   └── SendWelcomeEmail.php  
├── Notifications/  
│   ├── WelcomeNotification.php  
│   └── PasswordResetNotification.php  
└── Routes/  
    └── api.php
```

1. Routes — app/Modules/Auth/Routes/api.php

```
<?php
```

```
use App\Modules\Auth\Controllers\AuthController;
```

```
use Illuminate\Support\Facades\Route;
```

```
Route::prefix('auth')->name('auth.')->group(function () {
```

```
    // Public endpoints
```

```
    Route::post('/register', [AuthController::class, 'register'])->name('register');
```

```
    Route::post('/login', [AuthController::class, 'login'])->name('login');
```

```
    Route::post('/forgot-password', [AuthController::class, 'forgotPassword'])->name('forgot-password');
```

```
    Route::post('/reset-password', [AuthController::class, 'resetPassword'])->name('reset-password');
```

```
    // Protected endpoints
```

```
    Route::middleware('auth:sanctum')->group(function () {
```

```
        Route::get('/me', [AuthController::class, 'me'])->name('me');
```

```
        Route::post('/logout', [AuthController::class, 'logout'])->name('logout');
```

```
        Route::post('/logout-all', [AuthController::class, 'logoutAll'])->name('logout-all');
```

```
        Route::put('/profile', [AuthController::class, 'updateProfile'])->name('profile.update');
```

```
        Route::put('/password', [AuthController::class, 'updatePassword'])->name('password.update');
```

```
    });
```

```
});
```

2. DTOs

app/Modules/Auth/DTOs/RegisterDTO.php

```
<?php

namespace App\Modules\Auth\DTOs;

class RegisterDTO
{
    public function __construct(
        public readonly string $name,
        public readonly string $email,
        public readonly string $password,
        public readonly string $phone,
        public readonly string $businessName,
        public readonly string $timezone = 'Asia/Jakarta',
        public readonly string $currency = 'IDR',
    ) {}

    public static function fromRequest(array $data): static
    {
        return new static(
            name: $data['name'],
            email: $data['email'],
            password: $data['password'],
            phone: $data['phone'],
            businessName: $data['business_name'],
            timezone: $data['timezone'] ?? 'Asia/Jakarta',
            currency: $data['currency'] ?? 'IDR',
        );
    }
}
```

app/Modules/Auth/DTOs/LoginDTO.php

```
<?php
```

```
namespace App\Modules\Auth\DTOs;
```

```
class LoginDTO
```

```
{
```

```
    public function __construct(
```

```
        public readonly string $email,
```

```
        public readonly string $password,
```

```
        public readonly string $deviceName = 'web',
```

```
    ) {}
```

```
    public static function fromRequest(array $data): static
```

```
    {
```

```
        return new static(
```

```
            email:      $data['email'],
```

```
            password:   $data['password'],
```

```
            deviceName: $data['device_name'] ?? 'web',
```

```
        );
```

```
    }
```

```
}
```

3. Form Requests

app/Modules/Auth/Requests/RegisterRequest.php

```
<?php

namespace App\Modules\Auth\Requests;

use Illuminate\Foundation\Http\FormRequest;

class RegisterRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'name'          => ['required', 'string', 'max:100'],
            'email'          => ['required', 'email:rfc,dns', 'unique:users,email'],
            'password'       => ['required', 'string', 'min:8', 'confirmed'],
            'phone'          => ['required', 'string', 'max:20'],
            'business_name' => ['required', 'string', 'max:150'],
            'timezone'       => ['nullable', 'string', 'timezone:all'],
            'currency'       => ['nullable', 'string', 'size:3'],
        ];
    }

    public function messages(): array
    {
        return [
            'email.unique'      => 'Email sudah terdaftar.',
            'password.confirmed' => 'Konfirmasi password tidak cocok.',
            'timezone.timezone' => 'Timezone tidak valid.',
        ];
    }
}
```

app/Modules/Auth/Requests/LoginRequest.php

```
<?php
```

```
namespace App\Modules\Auth\Requests;
```

```
use Illuminate\Foundation\Http\FormRequest;
```

```
class LoginRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'email'      => ['required', 'email'],
            'password'   => ['required', 'string'],
            'device_name' => ['nullable', 'string', 'max:100'],
        ];
    }
}
```

app/Modules/Auth/Requests/ForgotPasswordRequest.php

```
<?php

namespace App\Modules\Auth\Requests;

use Illuminate\Foundation\Http\FormRequest;

class ForgotPasswordRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'email' => ['required', 'email', 'exists:users,email'],
        ];
    }

    public function messages(): array
    {
        return [
            'email.exists' => 'Email tidak ditemukan.',
        ];
    }
}
```

app/Modules/Auth/Requests/ResetPasswordRequest.php

```
<?php
```

```
namespace App\Modules\Auth\Requests;
```

```
use Illuminate\Foundation\Http\FormRequest;
```

```
class ResetPasswordRequest extends FormRequest
```

```
{
```

```
    public function authorize(): bool { return true; }
```

```
    public function rules(): array
```

```
    {
```

```
        return [
```

```
            'token' => ['required', 'string'],
```

```
            'email' => ['required', 'email', 'exists:users,email'],
```

```
            'password' => ['required', 'string', 'min:8', 'confirmed'],
```

```
        ];
```

```
    }
```

```
}
```


4. Resources

app/Modules/Auth/Resources/AuthResource.php

```
<?php

namespace App\Modules\Auth\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class AuthResource extends JsonResource
{
    private string $token;
    private string $tokenType;

    public function withToken(string $token, string $type = 'Bearer'): static
    {
        $this->token = $token;
        $this->tokenType = $type;
        return $this;
    }

    public function toArray(Request $request): array
    {
        return [
            'user' => [
                'id' => $this->id,
                'name' => $this->name,
                'email' => $this->email,
                'phone' => $this->phone,
                'avatar' => $this->avatar_url,
                'roles' => $this->getRoleNames(),
                'permissions' => $this->getPermissionNames(),
                'business' => $this->business ? [
                    'id' => $this->business->id,
                    'name' => $this->business->name,
                    'currency' => $this->business->currency,
                    'timezone' => $this->business->timezone,
                    'plan' => $this->business->subscription?->plan_name,
                ] : null,
                'created_at' => $this->created_at?->toIso8601String(),
            ],
        ];
    }
}
```

```
],
'token'      => $this->token      ?? null,
'token_type' => $this->tokenType ?? null,
'expires_at' => isset($this->token)
               ? now()->addDays(30)->toIso8601String()
               : null,
];
}
}
```

5. Service —

app/Modules/Auth/Services/AuthService.php

```
<?php
```

```
namespace App\Modules\Auth\Services;

use App\Modules\Auth\DTOs\LoginDTO;
use App\Modules\Auth\DTOs\RegisterDTO;
use App\Modules\Auth\Events\UserRegistered;
use App\Modules\Tenant\Models\Business;
use App\Shared\Exceptions\VeloraException;
use App\Shared\Services\BaseService;
use Illuminate\Auth\Events>PasswordReset;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades>Password;
use Illuminate\Support\Str;
use App\Models\User;

class AuthService extends BaseService
{
    /**
     * Register user + buat business (1 atomic transaction)
     */
    public function register(RegisterDTO $dto): array
    {
        return DB::transaction(function () use ($dto) {

            // 1. Buat user
            $user = User::create([
                'name'      => $dto->name,
                'email'     => $dto->email,
                'password' => Hash::make($dto->password),
                'phone'    => $dto->phone,
            ]);

            // 2. Buat business (tenant)
            $business = Business::create([
                'name'       => $dto->businessName,
                'owner_id'  => $user->id,
                'email'     => $dto->email,
                'currency' => $dto->currency,
```

```

        'timezone' => $dto->timezone,
    ]));

    // 3. Link user ke business
    $user->update(['business_id' => $business->id]);

    // 4. Assign default role
    $user->assignRole('owner');

    // 5. Dispatch event (setup branch, kirim email, dll via listener)
    event(new UserRegistered($user, $business));

    // 6. Buat token
    $token = $user->createToken('register', ['*'], now()->addDays(30))->plainTextToken;

    return ['user' => $user->load('business'), 'token' => $token];
});
}

/**
 * Login – rate limiting via throttle middleware di route
 */
public function login(LoginDTO $dto): array
{
    $user = User::where('email', $dto->email)->with('business')->first();

    if (!$user || !Hash::check($dto->password, $user->password)) {
        throw new VeloraException('Email atau password salah.', 401, 'INVALID_CREDENTIALS');
    }

    if (!$user->is_active) {
        throw new VeloraException('Akun Anda telah dinonaktifkan.', 403, 'ACCOUNT_DISABLED');
    }

    // Hapus token lama dengan device_name yang sama
    $user->tokens()->where('name', $dto->deviceName)->delete();

    $token = $user->createToken($dto->deviceName, ['*'], now()->addDays(30))->plainTextToken;

    return ['user' => $user, 'token' => $token];
}

/**

```

```

    * Logout – revoke current token
    */
public function logout(User $user): void
{
    $user->currentAccessToken()->delete();
}

/**
 * Logout all devices – revoke semua token
 */
public function logoutAll(User $user): void
{
    $user->tokens()->delete();
}

/**
 * Kirim reset password link
 */
public function sendResetLink(string $email): void
{
    $status = Password::sendResetLink(['email' => $email]);

    if ($status !== Password::RESET_LINK_SENT) {
        throw new VeloraException('Gagal mengirim link reset password.', 500, 'RESET_LINK_F/
    }
}

/**
 * Reset password dengan token
 */
public function resetPassword(string $token, string $email, string $password): void
{
    $status = Password::reset(
        ['token' => $token, 'email' => $email, 'password' => $password],
        function (User $user, string $password) {
            $user->forceFill([
                'password' => Hash::make($password),
                'remember_token' => Str::random(60),
            ]->save());

            // Cabut semua token lama setelah reset
            $user->tokens()->delete();
        }
    );
}

```

```

        event(new PasswordReset($user));
    }
);

if ($status !== Password::PASSWORD_RESET) {
    throw new VeloraException('Token reset password tidak valid atau kadaluarsa.', 422,
    }
}

/**
 * Update profile
 */
public function updateProfile(User $user, array $data): User
{
    $user->update(array_filter([
        'name' => $data['name'] ?? null,
        'phone' => $data['phone'] ?? null,
    ]));

    return $user->fresh();
}

/**
 * Update password
 */
public function updatePassword(User $user, string $currentPassword, string $newPassword): User
{
    if (!Hash::check($currentPassword, $user->password)) {
        throw new VeloraException('Password lama tidak sesuai.', 422, 'WRONG_CURRENT_PASSWORD');
    }

    $user->update([
        'password' => Hash::make($newPassword),
    ]);

    // Revoke semua token kecuali current
    $currentTokenId = $user->currentAccessToken()->id;
    $user->tokens()->where('id', '!=', $currentTokenId)->delete();
}
}

```

6. Controller —

app/Modules/Auth/Controllers/AuthController.php

```
<?php

namespace App\Modules\Auth\Controllers;

use App\Modules\Auth\DTOs\LoginDTO;
use App\Modules\Auth\DTOs\RegisterDTO;
use App\Modules\Auth\Requests\ForgotPasswordRequest;
use App\Modules\Auth\Requests\LoginRequest;
use App\Modules\Auth\Requests\RegisterRequest;
use App\Modules\Auth\Requests\ResetPasswordRequest;
use App\Modules\Auth\Resources\AuthResource;
use App\Modules\Auth\Services\AuthService;
use App\Shared\Controllers\BaseController;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class AuthController extends BaseController
{
    public function __construct(private readonly AuthService $authService) {}

    public function register(RegisterRequest $request): JsonResponse
    {
        $result = $this->authService->register(
            RegisterDTO::fromRequest($request->validated())
        );

        return $this->created(
            (new AuthResource($result['user']))->withToken($result['token']),
            'Registrasi berhasil. Selamat datang di VELORA!'
        );
    }

    public function login(LoginRequest $request): JsonResponse
    {
        $result = $this->authService->login(
            LoginDTO::fromRequest($request->validated())
        );

        return $this->success(
            (new AuthResource($result['user']))->withToken($result['token']),
```

```

        'Login berhasil.'
    );
}

public function me(Request $request): JsonResponse
{
    $user = $request->user()->load('business', 'business.subscription');

    return $this->success(new AuthResource($user), 'Data profil.');
```

}

```

public function logout(Request $request): JsonResponse
{
    $this->authService->logout($request->user());

    return $this->success(null, 'Logout berhasil.');
```

}

```

public function logoutAll(Request $request): JsonResponse
{
    $this->authService->logoutAll($request->user());

    return $this->success(null, 'Semua sesi berhasil diakhiri.');
```

}

```

public function forgotPassword(ForgotPasswordRequest $request): JsonResponse
{
    $this->authService->sendResetLink($request->validated('email'));

    return $this->success(null, 'Link reset password telah dikirim ke email Anda.');
```

}

```

public function resetPassword(ResetPasswordRequest $request): JsonResponse
{
    $data = $request->validated();
    $this->authService->resetPassword($data['token'], $data['email'], $data['password']);

    return $this->success(null, 'Password berhasil direset. Silakan login kembali.');
```

}

```

public function updateProfile(Request $request): JsonResponse
{
    $data = $request->validate([
```

```

        'name' => ['sometimes', 'string', 'max:100'],
        'phone' => ['sometimes', 'string', 'max:20'],
    ]);

    $user = $this->authService->updateProfile($request->user(), $data);

    return $this->success(new AuthResource($user), 'Profil berhasil diperbarui.');
```

}

public function updatePassword(Request \$request): JsonResponse
{
 \$data = \$request->validate([
 'current_password' => ['required', 'string'],
 'password' => ['required', 'string', 'min:8', 'confirmed'],
]);

 \$this->authService->updatePassword(
 \$request->user(),
 \$data['current_password'],
 \$data['password']
);

 return \$this->success(null, 'Password berhasil diperbarui.');
}
}

7. Event & Listener

app/Modules/Auth/Events/UserRegistered.php

```
<?php
```

```
namespace App\Modules\Auth\Events;
```

```
use App\Modules\Tenant\Models\Business;
```

```
use App\Models\User;
```

```
use Illuminate\Foundation\Events\Dispatchable;
```

```
use Illuminate\Queue\SerializesModels;
```

```
class UserRegistered
```

```
{
```

```
    use Dispatchable, SerializesModels;
```

```
    public function __construct(
```

```
        public readonly User $user,
```

```
        public readonly Business $business,
```

```
    ) {}
```

```
}
```

app/Modules/Auth/Listeners/SendWelcomeEmail.php

```
<?php

namespace App\Modules\Auth\Listeners;

use App\Modules\Auth\Events\UserRegistered;
use App\Modules\Auth\Notifications>WelcomeNotification;
use Illuminate\Contracts\Queue\ShouldQueue;

class SendWelcomeEmail implements ShouldQueue
{
    public string $queue = 'notifications';

    public function handle(UserRegistered $event): void
    {
        $event->user->notify(new WelcomeNotification($event->business));
    }
}
```

8. User Model Update — app/Models/User.php

```
<?php

namespace App\Models;

use App\Shared\Traits\HasUuid;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable, HasRoles, HasUuid;

    protected $fillable = [
        'id',
        'name',
        'email',
        'password',
        'phone',
        'business_id',
        'avatar',
        'is_active',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password'          => 'hashed',
            'is_active'         => 'boolean',
        ];
    }
}
```

```
/* — Relations — */
```

```
public function business(): \Illuminate\Database\Eloquent\Relations\BelongsTo
{
    return $this->belongsTo(\App\Modules\Tenant\Models\Business::class, 'business_id');
}
```

```
/* — Accessors — */
```

```
public function getAvatarUrlAttribute(): ?string
{
    return $this->avatar
        ? \Illuminate\Support\Facades\Storage::url($this->avatar)
        : null;
}
```

```
}
```

9. Migration — Users Table

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->uuid('business_id')->nullable()->index();
            $table->string('name', 100);
            $table->string('email', 150)->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('phone', 20)->nullable();
            $table->string('password');
            $table->string('avatar')->nullable();
            $table->boolean('is_active')->default(true)->index();
            $table->rememberToken();
            $table->timestamps();
            $table->softDeletes();

            $table->index(['email', 'is_active']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('users');
    }
};
```


10. Register Event di EventServiceProvider

app/Providers/EventServiceProvider.php

```
<?php

namespace App\Providers;

use App\Modules\Auth\Events\UserRegistered;
use App\Modules\Auth\Listeners\SendWelcomeEmail;
use Illuminate\Foundation\Support\Providers\EventServiceProvider as ServiceProvider;

class EventServiceProvider extends ServiceProvider
{
    protected $listen = [
        UserRegistered::class => [
            SendWelcomeEmail::class,
            // \App\Modules\Tenant\Listeners\SetupDefaultBranch::class,
            // \App\Modules\Subscription\Listeners\StartTrialSubscription::class,
        ],
    ];
}
```

Daftarkan di bootstrap/app.php :

```
->withProviders([
    App\Providers\EventServiceProvider::class,
    App\Providers\RepositoryServiceProvider::class,
])
```

11. Rate Limiting — bootstrap/app.php

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;

->withMiddleware(function (Middleware $middleware) {
    $middleware->statefulApi();
    $middleware->throttleApi();
})
->booted(function () {
    RateLimiter::for('login', function ($request) {
        return Limit::perMinute(5)->by($request->ip());
    });

    RateLimiter::for('forgot-password', function ($request) {
        return Limit::perMinute(3)->by($request->input('email'));
    });
});
```

Terapkan ke route:

```
Route::post('/login', [AuthController::class, 'login'])
    ->middleware('throttle:login')
    ->name('login');

Route::post('/forgot-password', [AuthController::class, 'forgotPassword'])
    ->middleware('throttle:forgot-password')
    ->name('forgot-password');
```

12. API Response Examples

Register — POST /api/v1/auth/register

Request:

```
{
  "name": "Ahmad Fauzi",
  "email": "ahmad@tokosaya.id",
  "password": "Rahasia123!",
  "password_confirmation": "Rahasia123!",
  "phone": "08123456789",
  "business_name": "Toko Saya",
  "timezone": "Asia/Jakarta",
  "currency": "IDR"
}
```

Response 201 :

```
{
  "success": true,
  "message": "Registrasi berhasil. Selamat datang di VELORA!",
  "data": {
    "user": {
      "id": "uuid-xxx",
      "name": "Ahmad Fauzi",
      "email": "ahmad@tokosaya.id",
      "roles": ["owner"],
      "business": {
        "id": "uuid-yyy",
        "name": "Toko Saya",
        "currency": "IDR",
        "plan": "trial"
      }
    },
    "token": "velora_xxxxxxxxxxxx",
    "token_type": "Bearer",
    "expires_at": "2026-06-18T05:14:00+07:00"
  },
  "meta": {
    "timestamp": "2026-05-19T05:14:00+07:00",
    "request_id": "req_abc123"
  }
}
```

Login — POST /api/v1/auth/login

Response 401 — Credentials salah:

```
{
  "success": false,
  "message": "Email atau password salah.",
  "meta": {
    "error_code": "INVALID_CREDENTIALS",
    "timestamp": "2026-05-19T05:14:00+07:00"
  }
}
```

Checklist Module Auth

- ✓ Register (user + business, atomic transaction)
- ✓ Login (rate-limited, device-based token)
- ✓ Logout (revoke current token)
- ✓ Logout All Devices
- ✓ Get Profile /me
- ✓ Update Profile
- ✓ Update Password
- ✓ Forgot Password (send reset link)
- ✓ Reset Password (verify token)
- ✓ Form Request validation per endpoint
- ✓ DTO (RegisterDTO, LoginDTO)
- ✓ AuthResource (user + token + business)
- ✓ Event: UserRegistered → SendWelcomeEmail (queued)
- ✓ Rate limiting (login 5/min, forgot-password 3/min)
- ✓ User Model dengan UUID + Sanctum + Spatie Roles
- ✓ Migration lengkap dengan index

Next: Module Tenant — Business registration, Branch management, Tenant isolation scope